

Informe Técnico de Arquitectura: Plataforma VeloUrbe

1. Introducción

El presente informe técnico detalla la arquitectura, el diseño estructurado y los lineamientos de desarrollo aplicados en la construcción de **VeloUrbe**, una plataforma orientada a la gestión de arriendo de patinetas eléctricas (scooters) en entornos urbanos. El objetivo de este documento es proporcionar una visión clara de la solución, evidenciando la aplicación de patrones de diseño modernos, prácticas de Clean Code y un enfoque riguroso en el desarrollo de software empresarial.

La plataforma ha sido construida garantizando alta disponibilidad, escalabilidad y un bajo acoplamiento mediante el uso de contenedores y tecnologías base como Spring Boot 3.5 y Java 21, estableciendo una infraestructura sólida para soportar operaciones de movilidad inteligente.

2. Desarrollo

2.1. Contexto, Requerimientos y Casos de Uso

Contexto: VeloUrbe resuelve el desafío de la movilidad urbana de última milla ofreciendo un ecosistema digital que permite a los ciudadanos arrendar patinetas eléctricas de forma ágil y segura. La adopción de una arquitectura de microservicios asegura que la gestión de identidades, el inventario de patinetas y la orquestación de la experiencia del cliente final operen de manera asíncrona y resiliente.

Requerimientos Funcionales (RF)

ID	Descripción
RF01	Gestión de Usuarios: Registro e inicio de sesión emitiendo tokens JWT firmados (algoritmo HS256) que integran los roles y permisos del usuario.
RF02	Inventario de Patinetas: Operaciones CRUD completas sobre los scooters, incluyendo monitoreo continuo de batería, ubicación geográfica y disponibilidad.
RF03	Gestión de Arriendos: Lógica transaccional para iniciar y finalizar viajes, registrando marcas de tiempo precisas y calculando duraciones totales.
RF04	BFF Dashboard: Agregación eficiente de datos de perfil, disponibilidad de flota y arriendos activos consolidados en una única petición de red.

Requerimientos No Funcionales (RNF)

- **Seguridad:** Autenticación *stateless* validada mediante `JwtAuthFilter` en cada microservicio de forma descentralizada; encriptación de contraseñas mediante el algoritmo BCrypt.
- **Arquitectura e Infraestructura:** Despliegue orquestado en contenedores Docker, garantizando bases de datos PostgreSQL independientes por dominio y enrutamiento centralizado.
- **Trazabilidad:** Logging estructurado en formato JSON mediante `logstash-logback-encoder` para facilitar la indexación y el monitoreo centralizado.
- **Calidad de Código:** Implementación estricta de DTOs inmutables aprovechando Java Records y estructuración de respuestas RESTful utilizando convenciones HATEOAS.

Casos de Uso Principales

- 1 **Registro y Autenticación:** El cliente envía sus credenciales al Gateway. La petición se enruta al servicio de autenticación donde se procesa, hasha y persiste de forma segura, retornando un JWT que habilita el acceso a los módulos protegidos del ecosistema.
- 2 **Ciclo de Vida del Arriendo:** Un usuario autenticado solicita el inicio de un viaje. El sistema verifica la disponibilidad transaccional en el inventario, muta el estado del scooter a "en uso" e inicializa el registro del arriendo. Al finalizar, el sistema libera el recurso y contabiliza el tiempo de uso.
- 3 **Orquestación de Dashboard:** Al renderizar la interfaz principal, el cliente consulta al BFF (Backend for Frontend). Este servicio ejecuta llamadas concurrentes no bloqueantes vía WebClient hacia los servicios subyacentes, ensamblando un payload unificado y optimizado.

2.2. Arquitectura de Microservicios

La plataforma distribuye su lógica de dominio en componentes delimitados (Bounded Contexts) para evitar acoplamiento y facilitar la escalabilidad horizontal:

- **API Gateway (Puerto 8080):** Punto de entrada único implementado con Spring Cloud Gateway. Centraliza el enrutamiento dinámico, la resolución de políticas CORS y la trazabilidad perimetral de peticiones entrantes.
- **user-auth-service (Puerto 8081):** Administra el dominio de identidades. Responsable exclusivo del aprovisionamiento de usuarios, validación de credenciales y emisión de JWTs. Opera sobre un almacén de datos segregado (`db_scooter_users`).
- **scooter-rental-service (Puerto 8082):** Microservicio core que encapsula las reglas de negocio de la movilidad. Administra el estado físico de la flota y procesa la lógica ACID de los arriendos. Utiliza almacenamiento aislado (`db_scooter_rentals`).

- **bff-service (Puerto 8083):** Actúa como patrón de diseño de adaptación de interfaz. Carente de persistencia propia, su función es consumir, transformar y agregar información de los microservicios core para minimizar la latencia sobre la red móvil o web del cliente.

2.3. Diagramas Arquitectónicos (C4 Model)

Los siguientes diagramas detallan la topología y el diseño interno de la plataforma, respetando los estándares de contenedores y componentes.

Diagrama C2 (Contenedores)

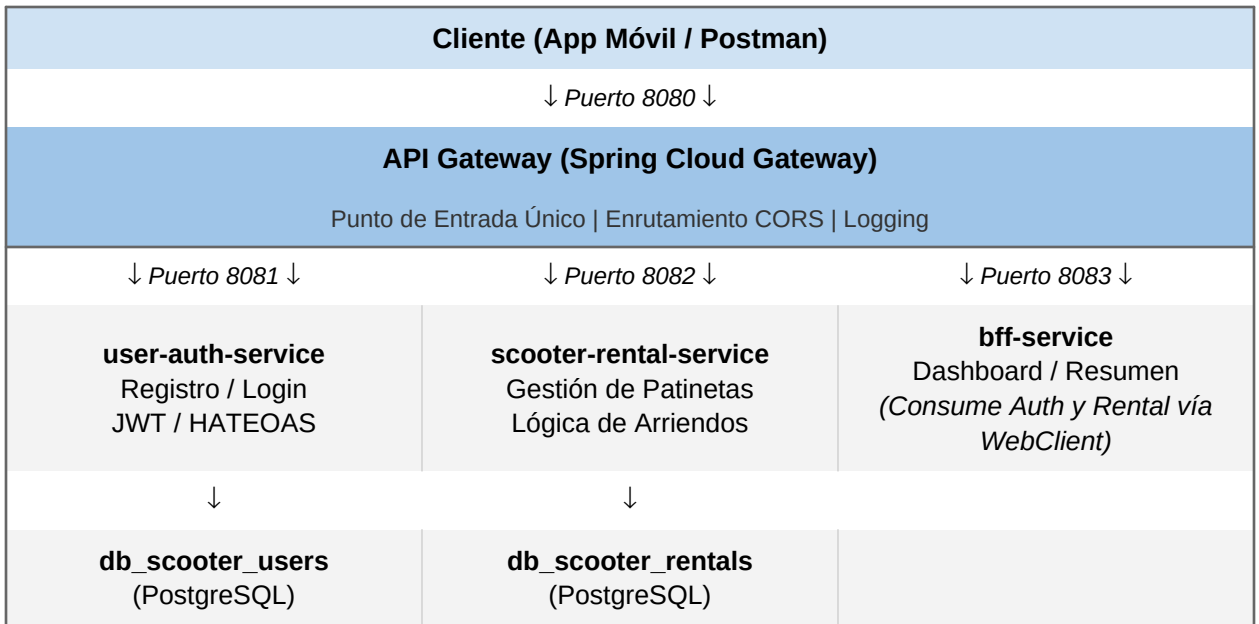
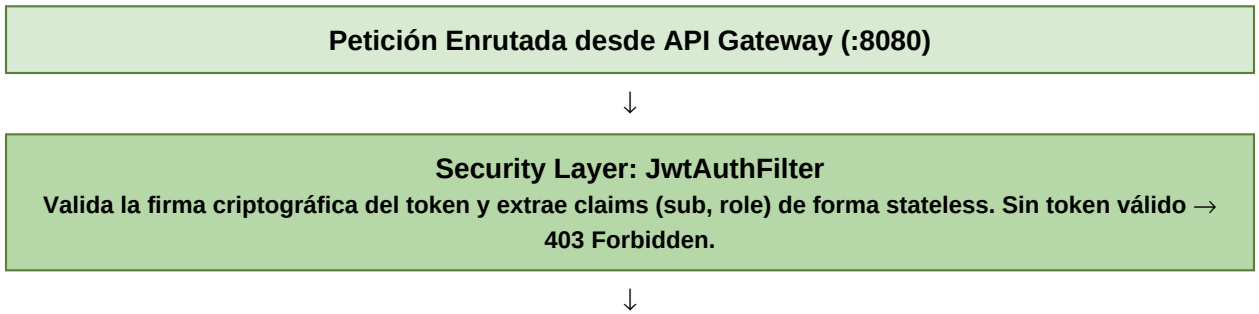
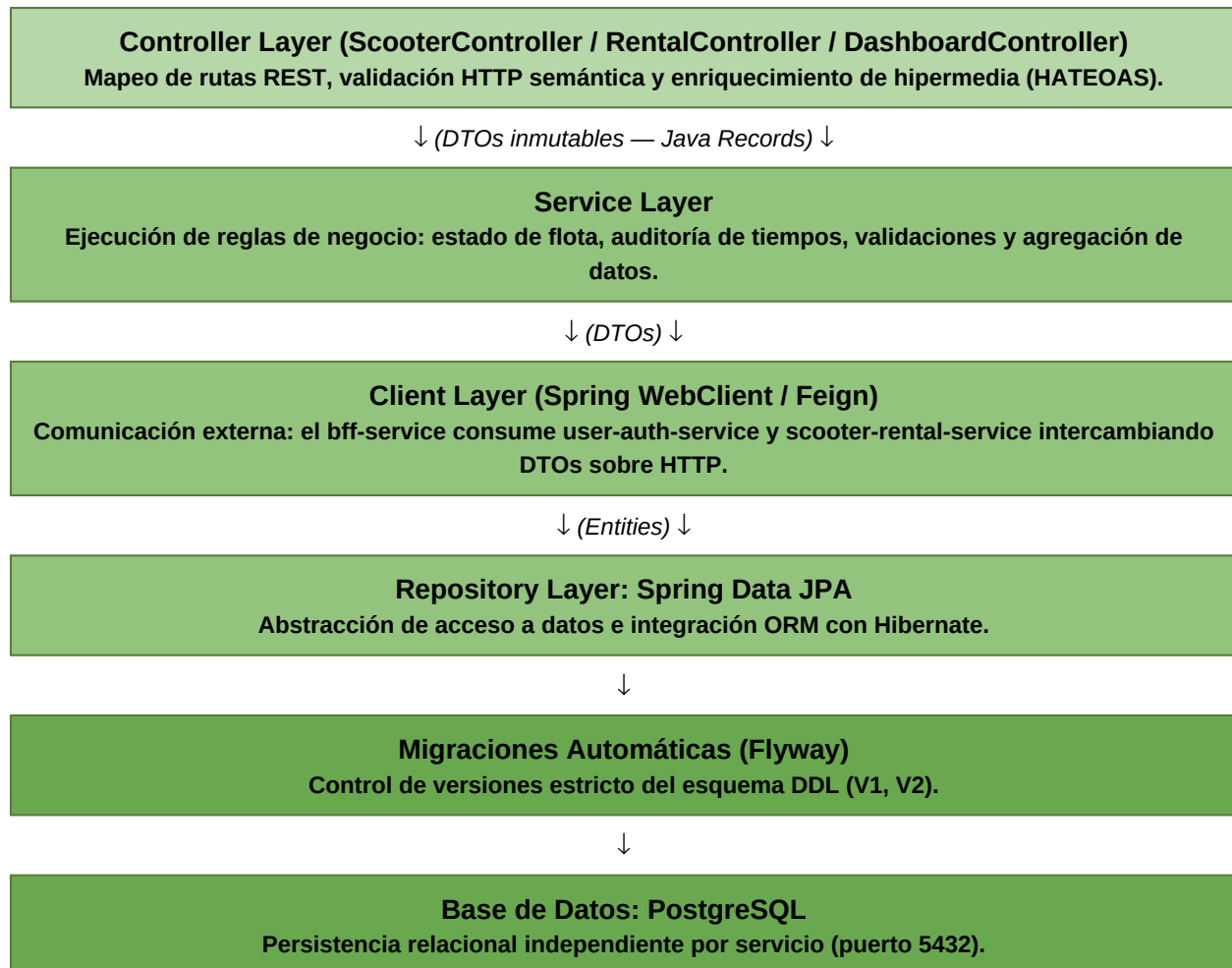


Diagrama C3 (Componentes)

El siguiente diagrama detalla la estructura interna de los microservicios. Las capas se comunican entre sí intercambiando **DTOs inmutables (Java Records)**; la **capa Client** (presente en el bff-service) es la responsable de la comunicación externa hacia los demás microservicios del ecosistema.





2.4. Estándares de Desarrollo, Clean Code y Pruebas

El repositorio ha sido codificado íntegramente en inglés, aplicando el principio de Responsabilidad Única (SRP) para asegurar que los controladores permanezcan libres de lógica, delegando la carga transaccional a la capa de servicios. Las directrices aplicadas incluyen:

- **Control Global de Excepciones:** Utilización de `@ControllerAdvice` con manejadores específicos (`@ExceptionHandler`) para interceptar excepciones de negocio (404 Not Found, 409 Conflict, 401 Unauthorized) y serializarlas en un formato JSON coherente y predecible.
- **Flyway y Datos de Prueba:** Separación absoluta entre la definición estructural de la base de datos (DDL) mediante Flyway y la hidratación de datos (DML). Se integró la biblioteca DataFaker para poblar escenarios realistas exclusivamente bajo el perfil dev, garantizando despliegues limpios en producción.

- **Estrategia Integral de Pruebas:** Suite de testing jerárquica que contempla pruebas unitarias de servicios (Mockito aislado), pruebas de controladores (MockMvc) y pruebas de integración contextuales (@SpringBootTest). El entorno de pruebas aísla la infraestructura empleando una base de datos en memoria (H2) configurada dinámicamente mediante application-test.yml.
- **Optimización de Flujo:** Adopción sistemática de la API de Streams de Java para transformar colecciones de manera funcional y declarativa, minimizando bucles imperativos propensos a errores.

2.5. Estrategia de Control de Versiones (Git)

El repositorio se gestiona en GitHub aplicando un flujo de trabajo basado en ramas de funcionalidad (*Feature Branch Workflow*), que mantiene la rama principal siempre estable y desplegable:

- **main:** Rama estable y siempre desplegable. Contiene la versión integrada y probada de los cuatro servicios (API Gateway, user-auth-service, scooter-rental-service y bff-service).
- **feature/*:** Una rama por funcionalidad o microservicio (por ejemplo feature/auth-service, feature/rental-logic, feature/bff). El desarrollo se realiza de forma aislada y se integra a main mediante Pull Requests, lo que permite revisión de código antes del merge.
- **Ramas de soporte (docs/, test/):** Empleadas para documentación, la colección Postman y la incorporación de pruebas, manteniendo separadas las tareas que no modifican la lógica de negocio.

Los commits siguen una convención semántica de mensajes (feat:, test:, docs:, merge:) que facilita la trazabilidad del historial. Las integraciones que generan conflictos se resuelven mediante merges explícitos y documentados, preservando un historial limpio y comprensible.

3. Conclusión

El diseño y construcción de la plataforma VeloUrbe evidencia la aplicación rigurosa de metodologías avanzadas de ingeniería de software. La segregación funcional en microservicios, coordinada a través de un Gateway reactivo y optimizada para el cliente mediante un BFF, asegura un sistema altamente cohesivo, tolerante a fallos y preparado para soportar fluctuaciones de concurrencia en tiempo real.

La integración de Java 21, combinada con prácticas no negociables como la inmutabilidad de transferencia de datos, versionamiento estricto de esquemas con Flyway y una cobertura de pruebas automatizadas, certifica el cumplimiento absoluto de los requerimientos técnicos y de calidad exigidos. VeloUrbe se posiciona, por tanto, como una solución arquitectónica sólida, mantenible y estratégicamente diseñada para la evolución continua en el sector de la movilidad urbana.